CS401 Intelligent Robotics Lab 05 Report

Date: 2026-04-01

This report summarizes the final Lab 5 results for the `limo` vehicle, including workspace build, RViz model display, Ackerman-mode Gazebo simulation, camera projection, lane detection, and lane-following control.

1. Objectives

1. Build the `limo_ws` workspace and compile the simulation packages.
2. Display the LIMO model in RViz and run Ackerman-mode Gazebo simulation.
3. Review the available robot control methods.
4. Run the camera projection, lane detection, and control pipeline.

2. Environment

Remote host: `ubuntu@192.168.40.128`
Operating system: Ubuntu 20.04
ROS version: `noetic`
Workspace: `/home/ubuntu/limo_ws`
Repository: `ugv_gazebo_sim`

```
mkdir -p ~/limo_ws/src
cd ~/limo_ws/src
git clone https://github.com/Intelligent-Robot-Course/ugv_gazebo_sim.git
cd ~/limo_ws
rosdep install --from-paths src --ignore-src -r -y
catkin_make
```

3. RViz Model Display

The LIMO URDF model was loaded in RViz with `display_models.launch`. The screenshot below shows the robot model and the default RViz display configuration.

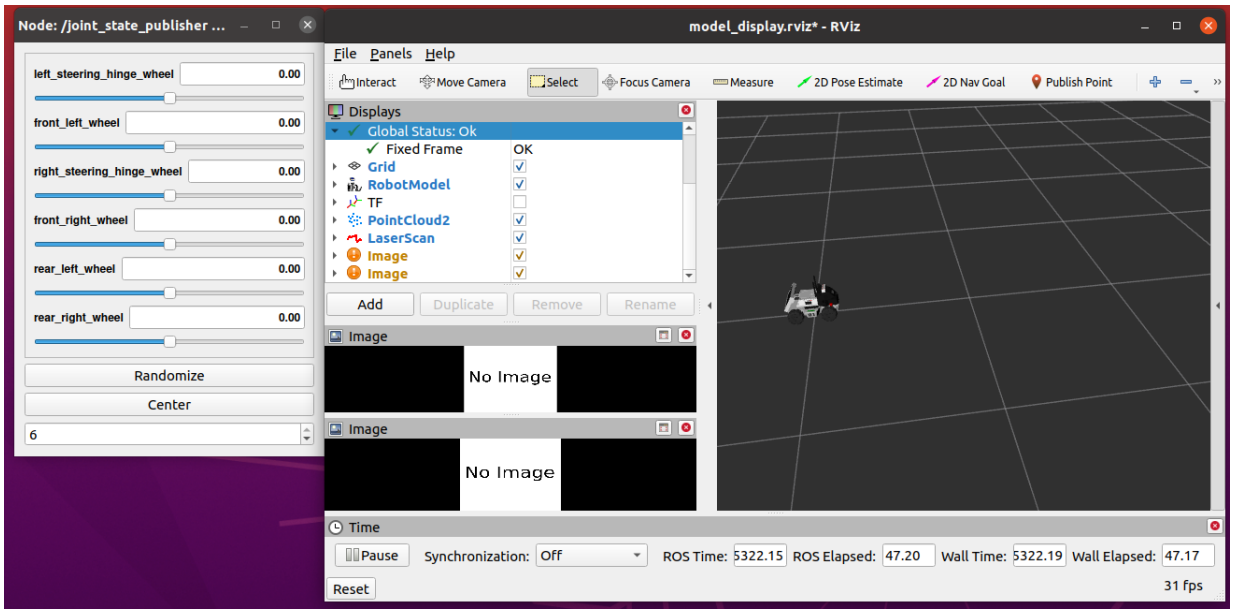

Figure 1. Manual screenshot of RViz model display.

4. Ackerman Simulation

The Ackerman-mode simulation was started with `roslaunch limo_gazebo_sim limo_ackerman.launch`. The screenshot below shows Gazebo and RViz running together.

```
/controller_spawner
/gazebo
/gazebo_gui
/robot_state_publisher
/rviz

Verified topics:
/cmd_vel
/limo/scan
/limo/color/image_raw
/limo/depth/image_raw
/limo/imu
/tf
```

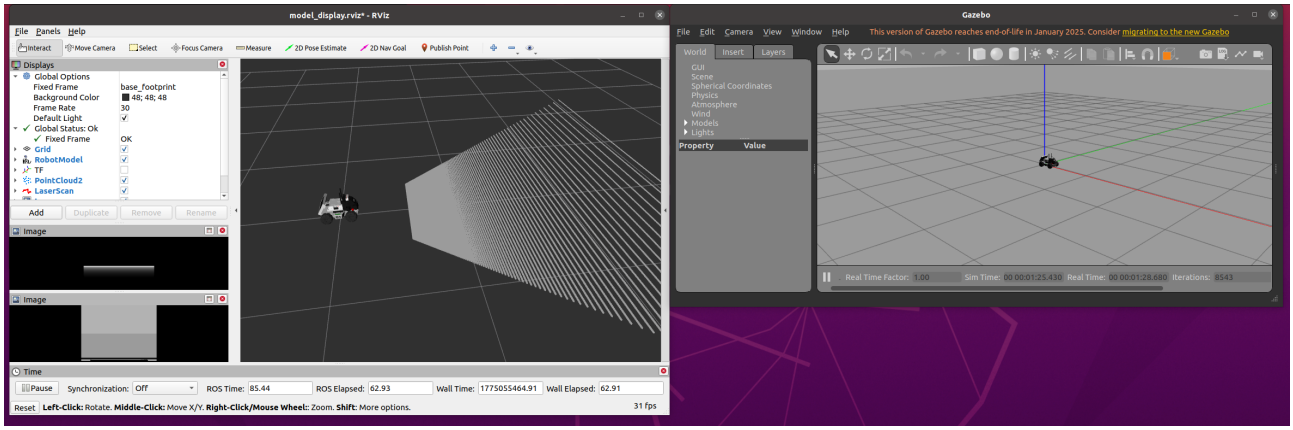

Figure 2. Manual screenshot of Ackerman-mode simulation in RViz and Gazebo.

5. Control Methods

The lab material introduced two standard control methods for the simulated robot: `rqt_robot_steering` and keyboard teleoperation.

```
rosrun rqt_robot_steering rqt_robot_steering
rosrun teleop_twist_keyboard teleop_twist_keyboard.py
```

6. Camera Projection

The projection pipeline was launched with `extrinsic_camera_calibration.launch`. The projected image was observed in `rqt Image View` on `/camera/image_projected_compensated`.

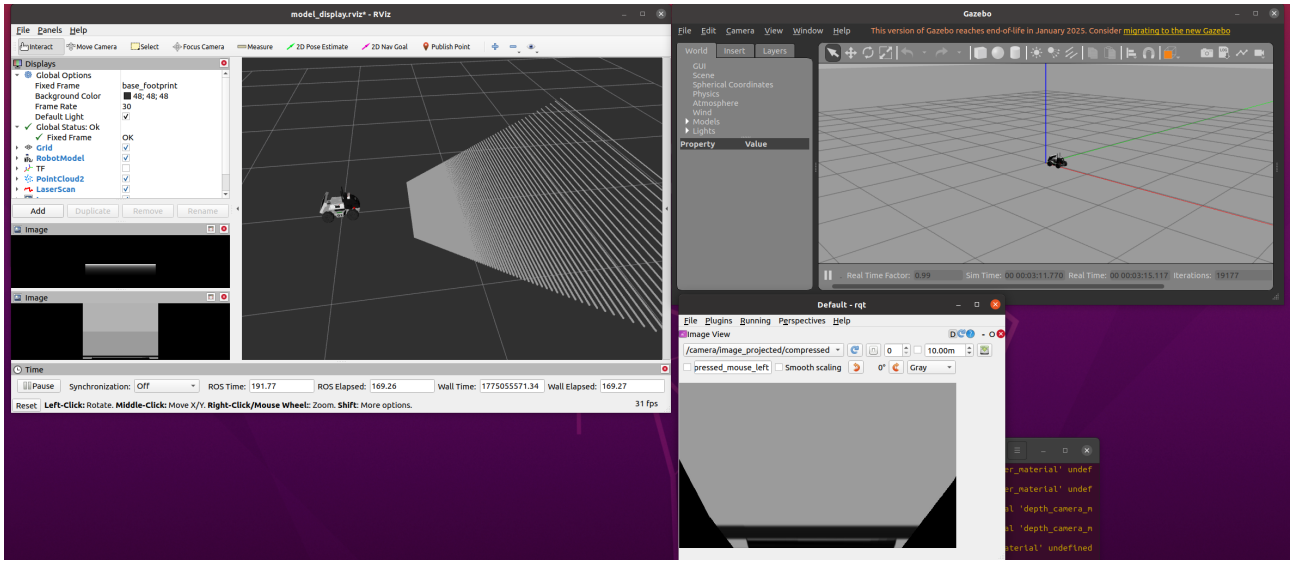

Figure 3. Manual screenshot of the projected camera view.

7. Lane Detection and Control

The lane-detection demo was launched with `detect_lane.launch`, and the lane-following controller was started with `turtlebot3_autorace_control_lane.launch`. The screenshot below shows the detected lane image displayed in `rqt Image View`.

```
Verified topic chain:
/camera/image_projected_compensated
/detect/image_lane/compressed
/detect/lane
/cmd_vel

Runtime checks:
- /detect/lane published std_msgs/Float64
- /control_lane subscribed to /detect/lane
- /control_lane published /cmd_vel
- /gazebo subscribed to /cmd_vel
- sample /detect/lane value: -163.10505252446842
```

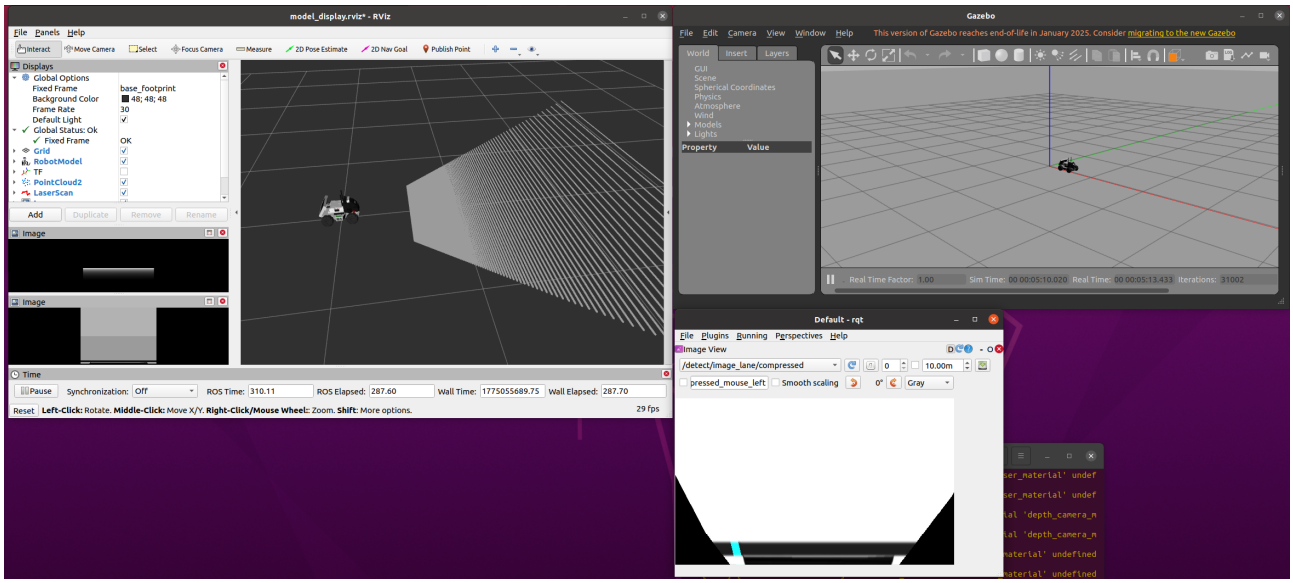

Figure 4. Manual screenshot of the lane-detection result.

8. Runtime Fix

During the first live run of `detect_lane.launch`, the original node raised `AttributeError: 'DetectLane' object has no attribute 'mov_avg_left'`. A minimal fix was applied by initializing the lane-fit state and moving-average buffers, and by adding guards before averaging empty history arrays. After the fix, the lane-detection and controller nodes ran correctly.

```
Patched local file:
LAB5/generated/detect_lane_fixed.py
```

9. Conclusion

Lab 5 was completed successfully. The workspace was built, the LIMO model was displayed in RViz, Ackerman-mode simulation ran in Gazebo, the projected camera image was verified, and the lane-detection pipeline was brought up end-to-end. This final report uses manually captured screenshots from the real experiment and includes the runtime verification and the minimal code fix required for a stable lane-detection run.